

Naming conventions for solution elements

A name is the smallest design decision in your program — and the one you make most often. Get it right and your code reads like plain English. Get it wrong and your own project, six weeks later, reads like a stranger's handwriting. This dot point names two conventions you must know: Hungarian notation and camelCase.

What this key knowledge covers

This summary distils everything you need for **KK09 — Naming conventions for solution elements** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Code is read far more than it is written

You type a line of code once. You — plus your teammates, your teacher, the marker of your SAT, and "future you" debugging at 11 pm — will *read* it dozens of times. The name is the part you read. A computer does not care what you call things; `x`, `a1` and `intHighScore` all run identically. Names are written entirely for *humans*.

□ The airport-luggage analogy

Imagine a baggage carousel where every suitcase is the same plain black bag with no tag. They all work as luggage — but finding *yours* means opening every single one. Now add tags: a colour for the airline, your name, your destination. You spot yours in a second. A naming convention is the agreed *system* for those tags, so everyone on the team reads them the same way.

Readability

A name should reveal purpose. `playerScore` beats `x` every time.

Consistency

One agreed system across the whole solution, so the team isn't reading five different styles.

Maintainability

Good names mean fewer comments, faster debugging, and cheaper changes months later.

In the exam, "purpose" answers nearly always come back to these three words: *readability*, *consistency*, *maintainability*. Lock them in.

What counts as a "solution element"?

The study design says naming conventions *distinguish solution elements*. So first — what are they? Meet **ArcadeBox**, a tiny retro arcade high-score app we'll use all the way through. Every kind of named thing appears in it.

Notice the prefixes already: txt btn lbl int bool. Those are Hungarian notation. The way each name runs words together with a capital — playerName, startGame — that's camelCase. They work *together*. We'll pull them apart next.

The Name-Tag Inspector

Pick a raw idea — say "player score" — then flip between conventions and watch the same element get re-tagged live. This is the single most useful drill for this dot point: see how one purpose is dressed in each style.

Hungarian: a type/control prefix is bolted on the front, then the rest is camel-cased. Best for telling similar elements apart.

Hungarian notation — a prefix that says what a thing is

Hungarian notation adds a short lowercase *prefix* to the front of a name to flag what kind of thing it is — its *type* or its *purpose*. The rest of the name describes what it's for. The combination of *type* + *purpose* in one name is the heart of the idea.

GUI control prefixes

txt text box → txtPlayerName btn button → btnStart lbl label → lblScore cbo combo box → cboDifficulty chk checkbox → chkSound

Data-type prefixes

int integer → intScore str string → strPlayerName bool boolean → boolGameOver dbl double → dblAvgScore arr array → arrHighScores

□ The backstory — why "Hungarian"?

It's named after **Charles Simonyi**, a Hungarian-born programmer who created the style at Xerox PARC and brought it to Microsoft in the 1980s, where it shaped early Word and Excel. The nickname is a small joke: Hungarian names are written *surname first* (family name, then given name) — and Simonyi's prefixes put the "type" first, before the descriptive part. So a name like strPlayerName reads "type-first", just like a Hungarian name. (Simonyi later became wealthy enough to fly to the International Space Station — twice — but that's a story for another lesson.)

Two flavours (worth a mark in extended answers)

Prefix shows the *data type*: intScore, strName. This is the version VCE uses and the one in most VB textbooks.

Prefix shows the *purpose/role*, not the raw type — e.g. rwPosition (row), cbWidth (count of bytes). Simonyi's original intent.

camelCase — humps where the words join

A name can't contain spaces, so we need a way to glue words together and still read them. camelCase keeps the *first* word lowercase, then capitalises the start of every word after it. The capitals stick up like a camel's humps — hence the name (coined in 1995, but in use since the 1970s).

player score → **playerScore**

Where you'll see it

camelCase is the standard for *variables and methods* in Visual Basic, Java, C#, JavaScript and many others. It also sits *inside* Hungarian names: in txtPlayerName, the txt is Hungarian and the PlayerName part is camel-cased.

Don't confuse it with its cousins

PascalCase is just camelCase with the *first* letter also capitalised. The exam tell: classes and objects (Player, ScoreBoard) use PascalCase; the variables and methods inside them use camelCase.

Live convention converter

Type any idea in plain words — "*final boss health bar*", "*is paused*", "*max attempts*" — and see it rendered in every convention at once. This is exactly the transformation an exam question expects you to do by hand.

Hungarian here uses a sensible default prefix; in real code you'd pick the prefix that matches the element's *actual* type or control.

Sort the names into the right convention

Drag each name into the bin for the convention it follows (or tap a chip, then tap a bin on touch devices). You'll get instant feedback and a score.

The P•E•L technique

Short-answer marks are won with structure, not waffle. Use **P•E•L**: *Point* (name it), *Explain* (how/why it works), *Link* (tie it to the scenario or a benefit). Draft an answer below to: "*Explain one benefit of using Hungarian notation when naming ArcadeBox's GUI controls.*" The meter lights up as your answer hits the ideas a marker looks for.

Point. A Hungarian prefix shows the type of each control directly in its name.

Explain. Naming the start button btnStart and the score label lblScore means a reader can tell a button apart from a label without opening the form designer.

Link. This improves *readability* and *maintainability* — anyone who later edits ArcadeBox can find and change the right control quickly, with less chance of wiring code to the wrong element.

VCE-style practice · 18 marks

Each question has a weak answer (with the marker's notes), a strong answer, and a marking guide. Try it on paper first, reveal, then self-mark — your total tracks in the dock below.

Cheat sheet

Names are for humans. Good ones buy readability, consistency and maintainability.

Variables, constants, functions/methods, objects/classes, GUI controls, files — anything you name.

Lowercase prefix = type/purpose (int str bool txt btn lbl) + descriptive body. Charles Simonyi, Microsoft, 1980s.

firstWordLowerThenHumps. Standard for variables & methods. Lives inside Hungarian names too.

PascalCase classes/objects · snake_case (Python) · SCREAMING constants.

Name a convention → always pair it with a *purpose*. Prefixes must not lie. Stay consistent.

Common traps & misconceptions

Exam trap

01 **"Hungarian notation = camelCase."** They're different things that *combine*. Hungarian = the meaningful *prefix* (type/purpose). camelCase = the *capitalisation pattern*. A Hungarian name is usually camel-cased after its prefix.

Exam trap

02 **Naming a convention but not its *purpose*.** "Define + give a benefit" questions need both. Always tie back to readability, consistency or maintainability.

Exam trap

03 **Spaces or illegal characters in a name.** player score and player-score aren't valid identifiers. That's the whole reason camelCase and snake_case exist.

Exam trap

04 **A prefix that lies.** strScore on something holding a number is worse than no prefix. The prefix must match the real type/control.

Exam trap

05 **Using PascalCase for a variable.** Variables and methods → camelCase; classes/objects → PascalCase. Mixing them up reads as a convention error.

Exam trap

06 **Vague names that are "valid" but useless.** intX, btnButton1. Convention followed, meaning lost. The descriptive part must describe.

Exam trap

07 **Inconsistency across the solution.** Half camelCase, half snake_case looks careless and costs "consistency" marks. Pick one system; apply it everywhere.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Justify · 2 marks

A student names a variable `x` to store a customer's total spend. Justify a better name and the convention it follows.

✗ A weak answer

You should call it total because `x` is bad. (→ *States a name but no convention and no real reason — 1.*)

✓ A strong answer

A clearer name is `customerTotalSpend`, written in **camel case** (first word lowercase, each later word capitalised). It is self-documenting — another programmer immediately knows what the variable holds — which improves readability and maintainability, whereas `x` conveys nothing about its purpose.

How the marks are awarded

- 1 mark — gives a meaningful name and names the convention (camel case).
- 1 mark — justifies it with readability/maintainability.

Q2. Describe · 2 marks

Describe **Hungarian notation** and give one example of a variable name that uses it.

✗ A weak answer

It's a way of naming things from Hungary. (→ *Misreads the term entirely — 0.*)

✓ A strong answer

Hungarian notation is a naming convention where a short prefix indicates the variable's **data type** or role. For example, `strSurname` shows the variable holds a string, and `intCount` shows it holds an integer. The prefix makes the variable's type clear wherever it is used.

How the marks are awarded

- 1 mark — describes the type-prefix idea.
- 1 mark — a valid example (e.g. `strSurname`, `intCount`, `blsPaid`).